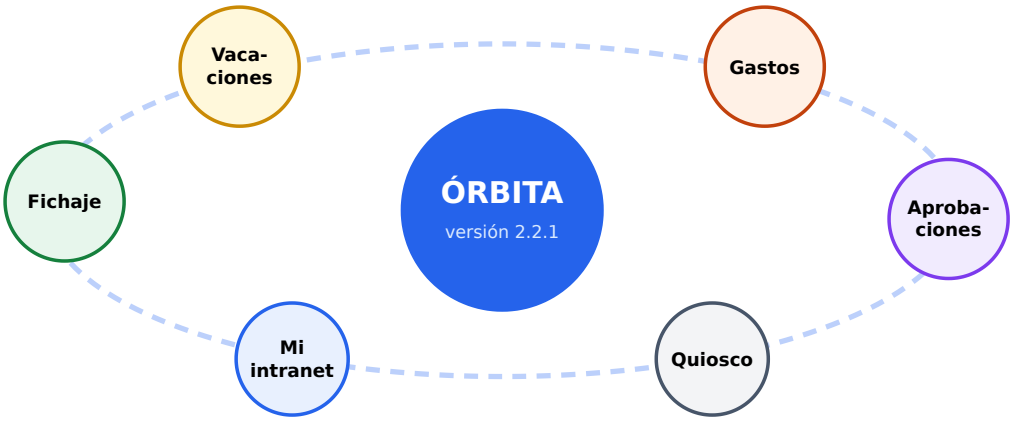
Las aplicaciones de ÓRBITA, explicadas fácil

Qué son, cómo están hechas por dentro, cómo han evolucionado de WebSphere Portal 8.5 a la versión 2.2.1 y cómo se instalan con Helm, paso a paso.

Repositorio
`https://github.com/europesip/orbita` · chart `oci://ghcr.io/europesip/charts/hcl-dx-orbita` · imágenes en `ghcr.io/europesip/orbita` (públicos)



Contenido

1 · Qué es ÓRBITA
2 · Usuarios de la demo y reglas
3 · Cómo se hizo: tres etapas
4 · El mapa: dónde vive cada pieza
5 · Por dentro: el código
6 · Quién es el usuario: lo dice DX
7 · La IA: el servidor MCP
8 · Cómo se construye y se publica
9 · Tutorial · instalar con Helm
10 · Conectar con IQ, comprobar y demo
11 · Actualizar, desinstalar, cambiar
12 · Historial de versiones
13 · Trampas y chuleta

ÓRBITA se instala sobre un HCL DX Compose con PostgreSQL. En el entorno de los labs (`github.com/europesip/hcl-dx-compose-labs`) es el Lab 5. Licencia PolyForm Noncommercial 1.0.0. Copyright © 2026 EUROPESIP.

1 · Qué es ÓRBITA

ÓRBITA es una **intranet de recursos humanos** para HCL DX Compose: los empleados fichan, piden vacaciones y pasan sus gastos, sus responsables lo aprueban, y un **asistente de IA** puede hacer todo eso hablando.

> **Analogía**
> Si DX Compose es el centro comercial, ÓRBITA es la **oficina de personal** que se instala dentro: tiene su ventanilla en el portal (las páginas), su oficina interna donde se hacen los trámites (la API) y su archivador (la base de datos PostgreSQL). Y desde la versión 2.1, un **asistente** que hace los trámites por ti si se lo pides.

Las seis aplicaciones

Fichaje
Un botón grande: Entrar · Salir · Volver. Pausas con motivo, oficina o teletrabajo, ubicación, línea de tiempo del día, registro mensual y correcciones. Cumple el registro de jornada (RDL 8/2019).

Vacaciones
Saldo del año (22 días + 2 de asuntos propios), el año entero en un calendario donde se piden días arrastrando, y el cuadrante del equipo.

Gastos y viajes
Notas de gasto con sus tickets (foto o PDF; el texto se lee solo con OCR), política por categoría, solicitudes de viaje y pagos por Administración.

Aprobaciones
La bandeja del responsable: ausencias, correcciones de fichaje, notas de gasto y viajes de su equipo, con cuánta gente falta esos días.

Mi intranet
La portada: fichaje rápido, horas de las últimas seis semanas, vacaciones, gastos por categoría, lo que tienes por aprobar, quién falta y los avisos.

Quiosco de fichaje
Una tablet en la entrada: cada persona ficha con su PIN o enseñando el QR de su tarjeta a la cámara.

Y lo que comparten todas

Función	Qué hace
Asistente “Pregúntale a ÓRBITA”	Chat con HCL DX IQ en un panel lateral. Actúa **como el usuario que pregunta**, con 26 herramientas: “¿cuántas vacaciones me quedan?”, “ficha mi entrada en teletrabajo”, “aprueba las vacaciones de macarena”. Responde en el idioma de la pregunta (español o inglés).
Avisos (campana)	Te avisa cuando alguien pide, aprueba, rechaza, comenta o paga algo tuyo.
Conversación	El responsable puede preguntar (“¿me pasas el ticket del peaje?”) sin rechazar la solicitud.
Lectura de tickets (OCR)	Al subir la foto se rellenan comercio, fecha, total, IVA y categoría. Se hace en tu navegador (Tesseract.js): la foto no sale a ningún servicio.

> **Idioma**
> La interfaz de las aplicaciones está en español; el asistente de IA entiende y responde en español o en inglés.

2 · Usuarios de la demo y reglas

Desde la versión 2.0 no se da de alta a nadie: la historia de la demo se escribe por **papeles**, y cada papel lo hace un **usuario de DX que ya existe**.

> **Analogía: una obra de teatro**
> El guion (los datos de la demo) tiene personajes: la persona de RRHH, el responsable de ventas, una comercial… El chart solo necesita saber **qué actor** (qué usuario de DX) hace cada papel. Eso es el valor `demo.people`. Por defecto, los actores son los usuarios del OpenLDAP de los labs.



Empleado
ficha, pide, consulta lo suyo

Responsable
+ ve su equipo y aprueba sus solicitudes

RRHH
+ toda la plantilla, exporta y restaura la demo

Administración
+ paga notas aprobadas y exporta a contabilidad

Papel	Usuario (labs)	Rol en ÓRBITA	PIN quiosco
`rrhh`	`patricia`	RRHH: ve a toda la plantilla, exporta, restaura la demo	1000
`ventas-responsable`	`andres`	Responsable de Ventas: aprueba a su equipo	2001
`ventas-1`	`macarena`	Comercial	1111
`ventas-2`	`eduardo`	Comercial	2222
`tecnologia-responsable`	`enrique`	Responsable de Tecnología	3001
`tecnologia-1`	`jordi`	Desarrollador	3333
`administracion`	`german`	Administración: paga las notas de gasto	6001
`tecnologia-2`, `-3`	(ficticios)	Aparecen en los datos; nadie entra con ellos	4444, 5555

> **Contraseñas**
> Las de esos usuarios en vuestro directorio (en los labs, las del OpenLDAP del Lab 2A). ÓRBITA no guarda contraseñas.

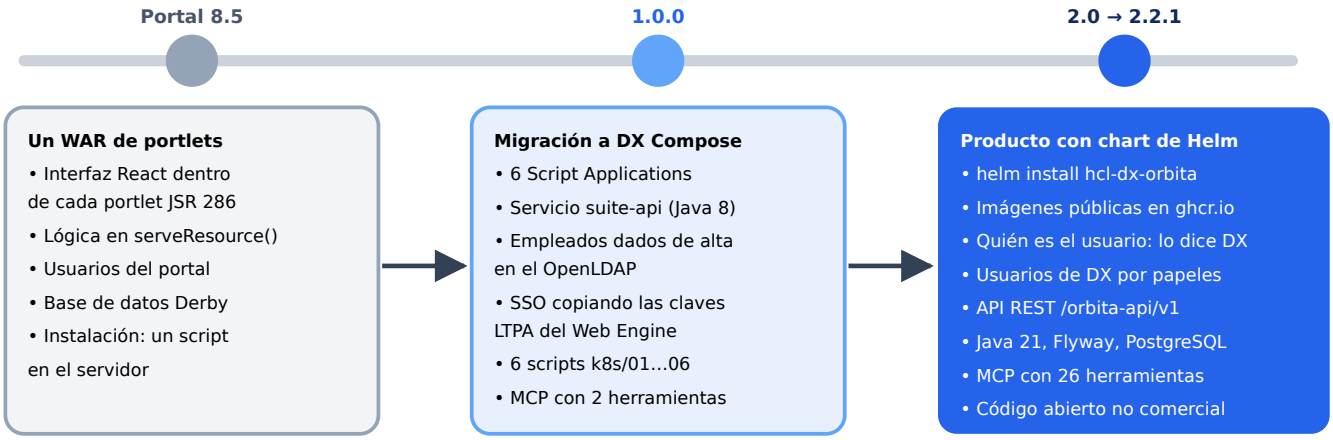
> **Volver a empezar la demo**
> RRHH › *Fichaje* › *Informes* › *Restaurar datos de demostración*. Los datos son relativos al día en que se cargan.

Reglas de negocio que conviene saber

Tema	Regla
Jornada teórica	Lunes a jueves 8 h, viernes 7 h (39 h semanales). Festivos nacionales cargados; RRHH añade los locales.
Fichajes	No se modifican ni se borran: un error se arregla con una **solicitud de corrección** que aprueba el responsable.
Pausas	Desayuno, comida, gestión personal: no cuentan como trabajo. Médico y visita a cliente: sí (retribuidas).
Política de gastos	Comidas hasta 30 €, alojamiento 120 €/noche, kilometraje 0,26 €/km, otros 50 €. Si se supera, hay que justificarlo.
Dirección	Quien no tiene responsable ve sus solicitudes aprobadas automáticamente.

3 · Cómo se hizo: tres etapas

ÓRBITA nació como portlets de WebSphere Portal 8.5. La versión 1.0 la llevó a DX Compose con scripts; la 2.0 la convirtió en un producto que se instala con un solo chart de Helm.



Analogía
En Portal 8.5 cada aplicación era un **todo en uno**: escaparate, oficina y archivo en la misma caja. En la 1.0 se separaron en piezas (escaparate en el portal, oficina aparte), pero el montaje era **artesanal**: siete scripts y muchos pasos a mano. En la 2.0 se convirtió en un **mueble de catálogo**: piezas fabricadas en serie (imágenes) y un único manual de montaje (el chart).

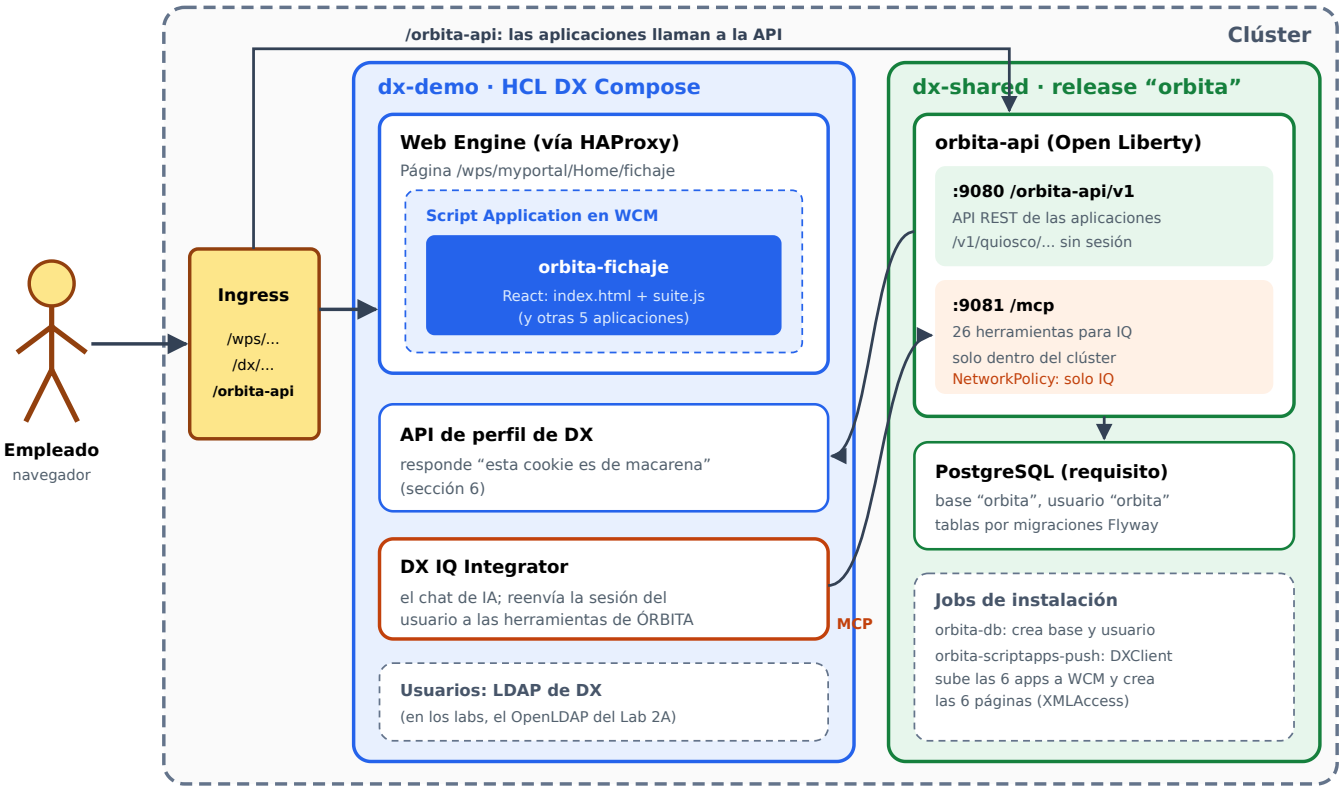
Qué cambió de la 1.0 a la 2.x

Tema	Versión 1.0	Versión 2.x
Instalación	Scripts `k8s/01` … `06`, compilación local, `kubectl cp`	Un chart: `helm upgrade --install`
Identidad	Claves LTPA copiadas, LDAP replicado	La API pregunta a DX quién es el usuario
Usuarios demo	`lucia.martin` … dados de alta en LDAP	Usuarios de DX existentes, por papeles
Base de datos	`suite_intranet`; un cambio de esquema borraba todo	`orbita` con migraciones Flyway: nunca se borran datos
API	`POST /suite-api/api` con `op=…`, siempre HTTP 200	`/orbita-api/v1` con métodos, rutas y códigos HTTP reales
Código	`portlet/java/com/demo/suite`, Java 8	`api/core|rest|mcp`, `com.europesip.orbita`, Java 21
IA	2 herramientas; el usuario era un parámetro	26 herramientas; actúa como el usuario de la sesión

Fuera los restos de Portal 8.5
La 2.0 eliminó la carpeta `portlet/java/com/demo/suite`, los descriptores `web.xml` e `ibm-web-bnd.xml`, Derby y Java 8. La interfaz React es la misma desde el principio: solo cambia la API a la que llama.

4 · El mapa: dónde vive cada pieza

ÓRBITA no es de HCL, así que se instala en el namespace de las herramientas de terceros (`dx-shared` en los labs), junto a PostgreSQL. En DX solo quedan sus páginas y sus Script Applications.



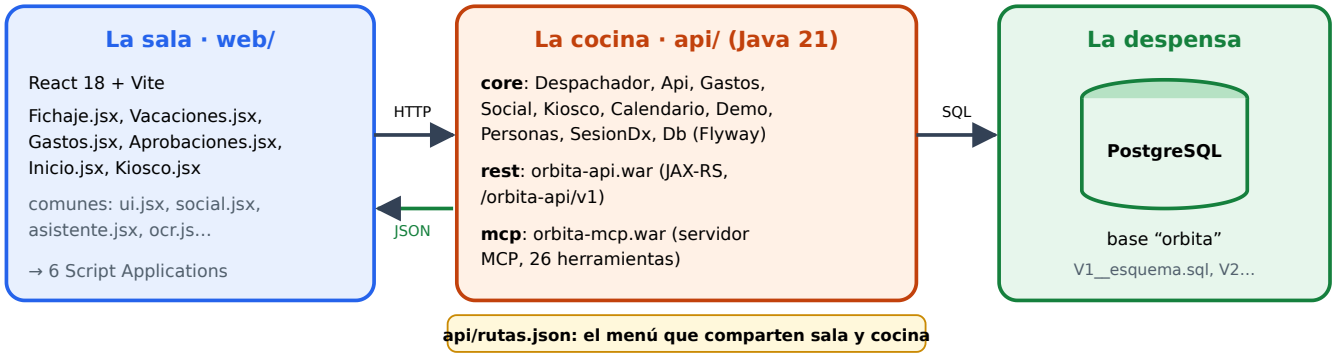
La interfaz se sirve desde DX; los datos los da orbita-api, que pregunta a DX quién es cada usuario; la IA (IQ) usa el MCP del mismo pod.

El truco del “mismo dominio”

Todo cuelga de `https://<host de DX>` : el portal en `/wps` , la API en `/orbita-api` y el chat de IQ en `/dx/api/iq` . Así el navegador envía la misma cookie de sesión a todo, sin problemas de CORS ni dobles logins.

5 · Por dentro: el código

Tres capas, como en un restaurante: la sala (lo que ves), la cocina (donde se hacen las cosas) y la despensa (donde se guarda todo). Y un **menú** que comparten sala y cocina: `api/rutas.json`.



Una API REST de verdad

`api/rutas.json` dice, para cada operación de la lógica, su método HTTP y su ruta. Lo leen el servidor y la interfaz, así que nunca se desincronizan:

```
GET    /orbita-api/v1/vacaciones/resumen          → saldo del año
POST   /orbita-api/v1/vacaciones/solicitudes      {"tipo":"VACACIONES","desde":"2026-12-
09","hasta":"2026-12-11"}
DELETE /orbita-api/v1/vacaciones/solicitudes/42
# 401 sin sesión de DX · 422 {"error":"No te quedan días","aviso":true} · 404 · 405 · 500
```

Familia	Rutas (debajo de `/orbita-api/v1/`)
General	`yo`, `inicio`, `config`, `salud`
Fichaje	`fichaje/hoy`, `fichaje/marcas`, `fichaje/meses/{año}/{mes}`, `fichaje/correcciones`, `fichaje/equipo`, `fichaje/informes/…`
Vacaciones	`vacaciones/resumen`, `vacaciones/solicitudes`, `vacaciones/calendario/…`, `vacaciones/anios/{año}`
Gastos y viajes	`gastos`, `gastos/notas/{id}`, `gastos/lineas`, `gastos/pagos`, `viajes` …
Aprobaciones	`aprobaciones`, `aprobaciones/{clase}/{id}`, `comentarios/{clase}/{objeto}`, `avisos`
Quiosco (pública)	`quiosco/identificacion`, `quiosco/marcas`: sin sesión, con la clave del dispositivo

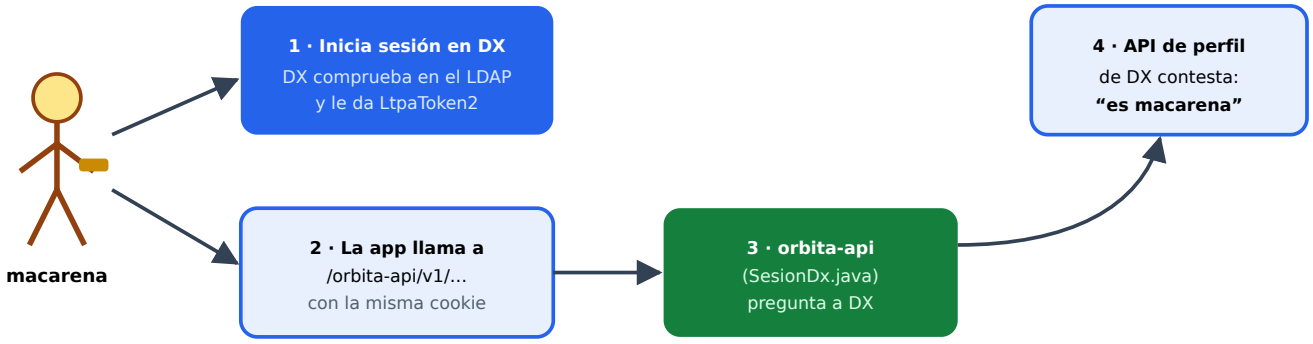
Un solo cerebro
La API REST y el servidor MCP llaman a la **misma** función, `Despachador.ejecutar(op, parámetros, usuario)`. Por eso la IA tiene exactamente los mismos permisos que el usuario en pantalla.

El esquema evoluciona sin perder datos
Las tablas las crean y actualizan **migraciones Flyway** (`V1__esquema.sql`, `V2__…`) al arrancar `orbita-api`. Una migración publicada nunca se edita: se añade otra.

6 · Quién es el usuario: lo dice DX

El empleado entra en el portal una vez y ÓRBITA ya sabe quién es. Desde la versión 2.0, ÓRBITA no lo averigua por su cuenta: **se lo pregunta a DX**.

Analogía: la pulsera del festival
Al entrar en el portal te ponen una **pulsera** (la cookie `LtpaToken2`). En la versión 1.0, ÓRBITA tenía una **copia del sello secreto** para comprobar pulseras por sí misma (claves LTPA copiadas). En la 2.0, cuando llegas con tu pulsera, ÓRBITA **llama a la taquilla** (la API de perfil de DX) y le pregunta "¿de quién es esta pulsera?". No guarda sellos, ni listas de asistentes, ni llaves de nadie.



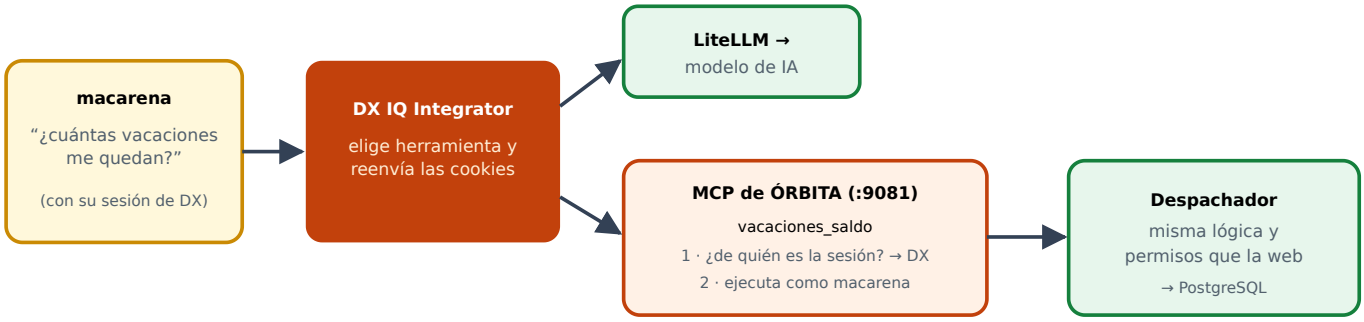
Ventaja	Por qué importa
Sin claves LTPA copiadas	En la 1.0 se copiaban las del Web Engine (iguales en todas las instalaciones de CF238: no aptas para producción). Ya no hacen falta.
Sin configuración de LDAP	ÓRBITA no conoce el directorio: vale cualquiera que use DX (LDAP, Active Directory…).
Sin credenciales guardadas	La API no guarda contraseñas de administrador de DX ni del bind del LDAP.
La IA no puede suplantar	El servidor MCP hace la misma pregunta con la sesión que reenvía IQ (sección 7).

El certificado de DX
Para hacer esa pregunta por HTTPS, la API tiene que fiarse del certificado de DX. Si es autofirmado o de una CA privada (como en los labs), se le da su CA en un Secret (`dx.caSecret`). `dx.insecureSkipTLSVerify` existe solo para laboratorio.

¿Y el quiosco?
Su página exige sesión (la tablet de la entrada inicia sesión con un usuario). Pero su parte de la API (`/v1/quiosco/…`) es pública: identifica a cada persona por su PIN o su QR, y al dispositivo por una clave que genera el chart (Secret `orbita-kiosk`).

7 · La IA: el servidor MCP

El botón “Pregúntale a ÓRBITA” (o el botón de IQ en DX) abre un chat con HCL DX IQ. Para hablar de vacaciones o gastos, ÓRBITA le presta un **maletín con 26 herramientas**: su servidor MCP.



Las 26 herramientas, por áreas

Área	Herramientas	≈ tokens
`general`	quien_soy, resumen_dia, avisos, avisos_marcar_leidos	470
`fichaje`	fichaje_hoy, fichaje_mes, equipo_ahora, **fichar**, fichaje_corregir	960
`vacaciones`	vacaciones_saldo, vacaciones_solicitudes, ausencias_equipo, vacaciones_solicitar, vacaciones_cancelar	910
`gastos`	gastos_resumen, nota_gasto, nota_gasto_crear, gasto_anadir, nota_gasto_enviar, viaje_solicitar, viaje_cancelar, pagos_pendientes	1.550
`aprobaciones`	aprobaciones_pendientes, **solicitud_resolver**, solicitud_comentarios, solicitud_comentar	780

Cada herramienta cuesta tokens
Su descripción viaja en **cada** llamada al modelo, se use o no. Las 26 son unas 4.700 (las de HCL, unas 30.000). Con `mcp.tools` se publican solo algunas áreas (`enabled`), se quitan otras (`disabled`) o solo las 14 de consulta (`readOnly: true`).

Fuera a propósito
Pagar, restaurar la demo, borrar y exportar no son herramientas de IA: son acciones masivas o irreversibles que no deberían quedar a una frase de distancia.

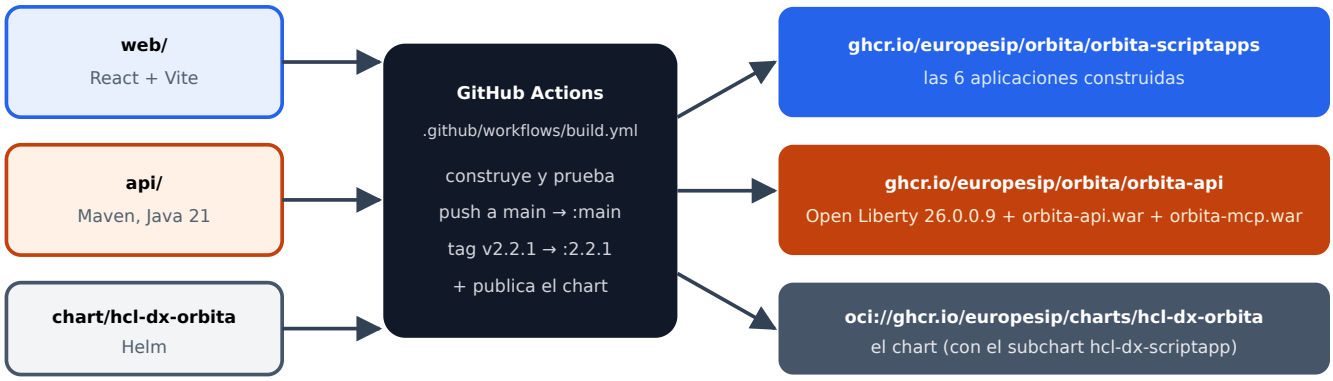
Seguridad

Capa	Cómo
Identidad	IQ reenvía las cookies de DX del usuario del chat y ÓRBITA pregunta a DX quién es. **No hay parámetro “usuario”**: el modelo no puede elegir en nombre de quién actúa, ni un texto malicioso puede hacerle suplantar a otro.
Red	El 9081 no sale por el Ingress; una NetworkPolicy solo deja entrar al Integrator.
Confirmación	Lo que modifica datos devuelve primero un resumen y solo actúa con `confirmar: true` .
Permisos y auditoría	Cada operación comprueba qué puede hacer esa persona, y cada llamada deja `ORBITA MCP auditoria herramienta=… usuario=…` en el log. Token Bearer opcional (`mcp.tokenSecret`).

Dos formas de poner IA en ÓRBITA
1. Con el Integrator de HCL DX IQ (lo que hace ÓRBITA hoy): el modelo configurado en IQ y todas sus herramientas a la vez, las de DX (WCM, DAM, páginas) y las de ÓRBITA. **2.** Con un chat propio: el MCP es estándar y la API está descrita en `rutas.json` ; la única condición es reenviar la sesión de DX (o un intercambio de identidad OAuth si el chat está fuera de DX).

8 · Cómo se construye y se publica

En la 1.0 se compilaba en tu máquina. Ahora lo hace **GitHub Actions**: construye, prueba y publica dos imágenes y el chart, todo público.



El repositorio

```
api/                  back end (Maven, Java 21, Jakarta EE 10)
  rutas.json          contrato de la API REST: método + ruta → operación
  core/               lógica (com.europesip.orbita.core): operaciones, permisos, Flyway, demo,
sesión de DX
  rest/               orbita-api.war: API REST (/orbita-api/v1)
  mcp/                orbita-mcp.war: servidor MCP
web/                  interfaz React (Vite); scripts/scriptapps.mjs genera las 6 Script Applications
images/               Dockerfiles: orbita-api (Open Liberty) y orbita-scriptapps
chart/hcl-dx-orbita/  chart de Helm
scripts/              ayudas para los labs: instalar, verificar, registrar el MCP en IQ
docs/                 especificación, MCP, preguntas a la IA, pruebas con REST Client
```

Para construir en local	Comando
Back end	`mvn -f api/pom.xml package` (Java 21)
Interfaz	`cd web && npm ci && npm run build:scriptapps`

La trampa de WCM con "[$"
WCM confunde `[$…]` con una etiqueta suya y la **borra** al servir el JavaScript, lo que rompe el código (`[$,H]=useState()`). `web/scripts/scriptapps.mjs` lo escribe con escapes equivalentes.

Otra rareza: Flyway en Open Liberty
Flyway solo encuentra las migraciones si su carpeta tiene un fichero `flyway.location`. Por eso existe en `db/migration`.

9 · Tutorial · instalar con Helm

Dos secrets, un fichero de valores y un comando

Antes de empezar

Necesitas	Detalle
HCL DX Compose	CF239 en Kubernetes con un Ingress (ingress-nginx) para su host, y un usuario **administrador** de DX para DXClient.
PostgreSQL	**Obligatorio en la 2.x** (probado con PostgreSQL 18 / CloudNativePG) y un Secret con un **superusuario** (`username` , `password`). En los labs: `external-deps/02-postgres.sh` (~3 min) o el Lab 2B.
Usuarios	Los de `demo.people` (en los labs, los del OpenLDAP del Lab 2A).
HCL DX IQ (opcional)	Para el asistente (Lab 3 de los labs).
Herramientas	`kubectl` y Helm 3.8 o superior. Imágenes y chart públicos: sin login.

1 **Secrets** (una vez): el administrador de DX para DXClient y la CA del certificado de DX.

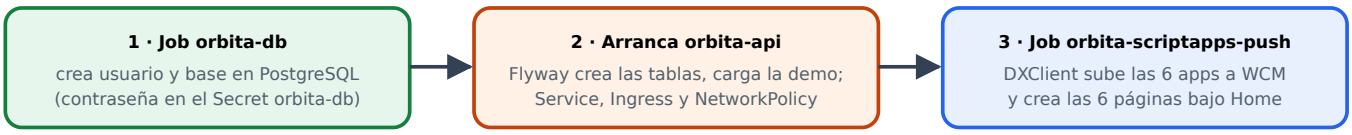
```
NS=dx-shared
kubectl -n $NS create secret generic orbita-dx --from-literal=username=wpsadmin --from-literal=password='<contraseña>'
kubectl -n dx-demo get secret dx-tls-cert -o jsonpath='{.data.tls\.crt}' | base64 -d > dx-ca.crt
kubectl -n $NS create secret generic orbita-dx-ca --from-file=ca.crt=dx-ca.crt
```

2 **Valores** (`orbita-values.yaml`):

```
dx:         { host: dx-demo.apps.promox.europesip-lab.com, caSecret: orbita-dx-ca }
database:   { host: dx-pg-rw.dx-shared.svc.cluster.local, create: { superuserSecret: dx-pg-superuser } }
mcp:        { networkPolicy: { integratorNamespace: dx-demo } }
scriptapps:                                  # el subchart no ve lo de arriba: se repite
  dx:       { host: dx-demo.apps.promox.europesip-lab.com, credentialsSecret: orbita-dx, caSecret: orbita-dx-ca }
  dxclient: { acceptLicense: true }          # aceptas la licencia de HCL DXClient
```

3 **Instalar**:

```
helm upgrade --install orbita oci://ghcr.io/europesip/charts/hcl-dx-orbita --version 2.2.1 \
  -n dx-shared -f orbita-values.yaml --timeout 15m
```



En los labs, con un script

Lee el `env.sh` de los labs y crea los secrets solo:

```
DXCLIENT_ACCEPT_LICENSE=yes scripts/instalar-en-labs.sh
scripts/verificar.sh
```

10 · Conectar con IQ, comprobar y demo

Registrar el MCP en IQ (para el asistente)

El Integrator de IQ tiene que conocer la URL base `http://orbita-api.dx-shared.svc.cluster.local:9081`. HCL documenta la variable `MCP_SERVER_LIST` para eso, pero **no tiene efecto** (caso de HCL CS1479102, sigue en CF239 / IQ 1.0.3): el Integrator lee otra lista, la del ConfigMap `dx-iq-integrator`. Mientras HCL lo corrige, se parchea esa lista:

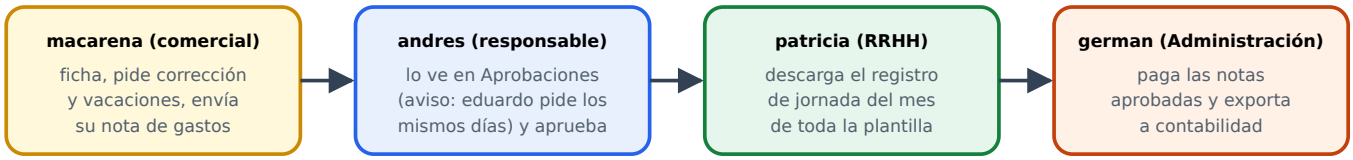
```
scripts/registrar-mcp-en-iq.sh       # en los labs: añade ÓRBITA a la lista, reinicia y muestra:
#   Fetched 26 tools from http://orbita-api.dx-shared.svc.cluster.local:9081
```

> **Tras cada `helm upgrade` de IQ, otra vez**
> El chart de IQ regenera la lista y ÓRBITA desaparece. Con Helm 4 ese upgrade necesita además `--force-conflicts`. Y tras cambiar `mcp.tools`, reinicia el Integrator para que lea la lista nueva.

Comprobar

Comprobación (`scripts/verificar.sh`)	Qué significa si falla
Release `orbita` desplegada	El `helm install` no terminó.
Job `orbita-db` completado	Revisa el host de PostgreSQL y el Secret de superusuario.
`orbita-api` lista y Flyway al día	Mira `kubectl -n dx-shared logs deploy/orbita-api`.
Job `orbita-scriptapps-push`: 6 apps en WCM	Credenciales de DX, CA o licencia de DXClient.
`/orbita-api/v1/salud` → 200; `/v1/yo` sin sesión → 401	Ingress o certificado.

Recorrido de la demo



Área	Prueba en el chat de IA, por ejemplo
Mi día	«¿Qué tengo hoy?» · «¿Tengo algo pendiente de aprobar?»
Fichaje	«Ficha mi entrada en teletrabajo» · «¿Qué días de este mes no cerré el fichaje?»
Vacaciones	«¿Cuántos días de vacaciones me quedan?» · «Pídeme vacaciones del 9 al 11 de diciembre»
Gastos	«Crea una nota “Visita a Sevilla” y añade una comida de ayer de 42,80 €» · «Envíasela a mi responsable»
Aprobaciones	Como andres: «Aprueba las vacaciones de macarena» · «Rechaza el viaje a Barcelona: presupuesto más ajustado»

Más de 60 preguntas de ejemplo en `docs/preguntas-ia.md` (en inglés, `docs/ai-questions.md`).

11 · Actualizar, desinstalar y cambiar cosas

Actualizar

```
helm upgrade orbita oci://ghcr.io/europesip/charts/hcl-dx-orbita --version <nueva> -n dx-shared -f
orbita-values.yaml
```

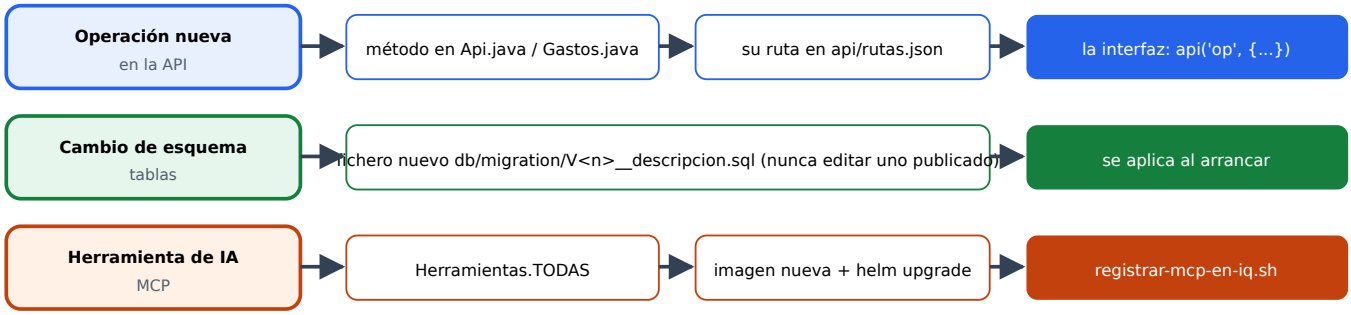
Las migraciones nuevas se aplican al arrancar y **los datos se conservan**. Las Script Applications se actualizan en sus páginas.

Desinstalar

```
helm -n dx-shared uninstall orbita
```

Se quedan a propósito: la base de datos y su Secret `orbita-db`, las Script Applications y las páginas (puede que alguien las use). Bórralas a mano si ya no las necesitas, y quita ÓRBITA de la lista de MCP de IQ con el mismo parche, sin su URL.

Cambiar cosas (desarrollo)



Ejemplo: una herramienta de IA

```
new Herramienta("fichaje_hoy", "Fichaje de hoy",
    "Indica si el usuario ha fichado hoy, desde qué hora y cuánto lleva trabajado. Úsala para '¿he
fichado?'.",
    Esquema.nuevo(), true,                    // true = solo consulta; sin parámetro de usuario
    l -> l.api("fichaje.estado")),
```

Regla	Por qué
Nombre en `snake_case`	IQ solo admite letras, números, `_` y `-`.
Buena descripción	Es lo único que lee el modelo para decidir si la usa (y viaja en cada petición).
`Esquema.nuevo()`	No hay parámetro de usuario: actúa siempre como el usuario de DX que pregunta.
Si modifica datos	`soloLectura = false`, `.confirmar()` y `l.pedirConfirmacion("resumen")` mientras no esté confirmado.

Probar el MCP a mano

`kubectl -n dx-shared port-forward svc/orbita-api 9081:9081` (tiene que ser el 9081 en local) y `docs/mcp-pruebas.http` con la cookie `LtpaToken2` de un usuario. Para Claude Code, solo en desarrollo con `ORBITA_MCP_DEV_USER` (el chart nunca la pone).

12 · Historial de versiones

Resumen del `CHANGELOG.es.md` del repositorio. Todas las versiones son del 6 de noviembre de 2026.

2.2.1 Preguntar en inglés o en español

- El panel “Pregúntale a ÓRBITA” responde en el idioma de cada pregunta (antes, siempre en español), y el MCP indica al modelo que responda en el idioma del usuario y traduzca los datos.
- Ejemplos en inglés en la documentación en inglés (README, Lab 5).

2.2.0 Elegir las herramientas MCP

- `mcp.tools` en el chart: publicar solo algunas áreas o herramientas (`enabled`), quitar algunas (`disabled`) o solo las de consulta (`readOnly`). Una herramienta no publicada tampoco se puede llamar.
- Instalación sin iniciar sesión en el registro (imágenes y chart públicos), y Lab 5 en los labs de HCL DX Compose.

2.1.0 La IA hace mucho más

- Servidor MCP con **26 herramientas** (antes 2): mi día y avisos, fichaje (también fichar), vacaciones, gastos y viajes (crear notas, añadir gastos, enviarlas) y aprobaciones (aprobar, rechazar, comentar). Lo que cambia datos pide confirmación.
- Más de 60 preguntas de ejemplo (`docs/preguntas-ia.md` , `docs/ai-questions.md`) y las dos formas de poner IA en ÓRBITA.
- Repositorio, imágenes y chart **públicos**.

2.0.0 Chart de Helm

Área	Cambios
Instalación	Chart `hcl-dx-orbita` en lugar de los scripts `k8s/01–06` : Job para la base, Deployment para la API y subchart `hcl-dx-scriptapp` 0.2.0 para las apps y páginas. Imágenes `orbita-api` (Open Liberty 26.0.0.9, Java 21) y `orbita-scriptapps` construidas por CI. Necesita PostgreSQL.
Usuarios y seguridad	Quién es el usuario lo dice DX (API de perfil): sin claves LTPA copiadas, sin LDAP replicado, sin credenciales de administrador. El MCP actúa como el usuario del chat (desaparece el argumento `usuario`). Usuarios de demo por papeles (`demo.people`). Clave del quiosco generada por el chart.
Datos	Migraciones Flyway: una actualización nunca borra datos. Solo PostgreSQL (fuera Derby). Base y usuario `orbita` . Restaurar la demo hace `TRUNCATE` y recarga.
Código y API	Repositorio reorganizado (`api/` , `web/` , `images/` , `chart/` , `docs/`), paquete `com.europesip.orbita` . API REST `/orbita-api/v1` con códigos HTTP reales, descrita en `api/rutas.json` . Fuera los restos de Portal 8.5. Páginas y clave del quiosco vienen de `GET /v1/config` .
Licencia	PolyForm Noncommercial 1.0.0.

1.0.0 Primera versión para DX Compose

- La suite de Portal 8.5 migrada a HCL DX Compose CF238 sin portlets: seis Script Applications y el servicio `suite-api` (Open Liberty 26.0.0.8).
- Instalación con los scripts `k8s/01-construir.sh` … `06-mcp-iq.sh` ; empleados dados de alta en el OpenLDAP (`lucia.martin` …); base `suite_intranet` ; SSO con claves LTPA copiadas; primer MCP con 2 herramientas.

13 · Trampas y chuleta

Trampa	Qué pasa y qué hacer
IQ pierde ÓRBITA	Tras cada `helm upgrade` de IQ: repetir `scripts/registrar-mcp-en-iq.sh` (bug CS1479102 de HCL).
Cambias `mcp.tools` y no se nota	El Integrator lee las herramientas al arrancar: `kubectl -n dx-demo rollout restart deployment/dx-iq-integrator`.
WCM borra `[$…]`	Rompería el JavaScript; `scriptapps.mjs` lo escapa.
Certificado de DX autofirmado	La API no puede preguntar quién es el usuario: dale la CA con `dx.caSecret`.
`/probe/ready` da 503	Normal hasta que `orbita-api` termina las migraciones.
Túnel al MCP con otro puerto	El virtual host solo acepta `Host: …:9081`: el puerto local tiene que ser 9081.
Probar `vacaciones_solicitar`	Con `confirmar: true` crea una solicitud real: cancélala o restaura la demo.
OCR de tickets	El navegador necesita internet la primera vez para bajar el motor y el modelo de español.

Chuleta

```
# ── Instalar (requisitos: DX Compose, PostgreSQL + Secret de superusuario) ──
kubectl -n dx-shared create secret generic orbita-dx --from-literal=username=wpsadmin --from-
literal=password='…'
kubectl -n dx-shared create secret generic orbita-dx-ca --from-file=ca.crt=dx-ca.crt
helm upgrade --install orbita oci://ghcr.io/europesip/charts/hcl-dx-orbita --version 2.2.1 \
  -n dx-shared -f orbita-values.yaml --timeout 15m

# ── En los labs ──────────────────────────────────────────────────────
DXCLIENT_ACCEPT_LICENSE=yes scripts/instalar-en-labs.sh
scripts/registrar-mcp-en-iq.sh        # asistente de IA (opcional)
scripts/verificar.sh

# ── Mirar ────────────────────────────────────────────────────────────
kubectl -n dx-shared logs job/orbita-scriptapps-push -c dxclient | tail -3
curl -k https://<host de DX>/orbita-api/v1/salud                # → ok
kubectl -n dx-shared logs deploy/orbita-api | grep "ORBITA MCP"    # herramientas y auditoría
kubectl -n dx-demo logs deploy/dx-iq-integrator | grep Fetched    # Fetched 26 tools ...

# ── Actualizar / desinstalar ─────────────────────────────────────────
helm upgrade orbita oci://ghcr.io/europesip/charts/hcl-dx-orbita --version <nueva> -n dx-shared -f orbita-
values.yaml
helm -n dx-shared uninstall orbita
```

Las 4 ideas para recordar

1. La interfaz son 6 **Script Applications** en WCM.
2. La lógica es **orbita-api**, instalada con **un chart**.
3. **DX dice quién es el usuario**: ÓRBITA no guarda claves.
4. La IA usa la **misma lógica y permisos** vía MCP.

Dónde seguir

`github.com/europesip/orbita`
`README.es.md` · `CHANGELOG.es.md`
`docs/mcp.md` · `docs/preguntas-ia.md`
`chart/hcl-dx-orbita/values.yaml`
Servicios: `www.europesip.com`

Licencia: PolyForm Noncommercial 1.0.0. Se puede usar, estudiar, modificar y redistribuir con fines no comerciales; el uso comercial requiere un acuerdo con EUROPESIP.